

VHDL Programming: UNIT 1

Why VHDL important

- **Geographical Preferences:**

The choice between VHDL and Verilog can sometimes be influenced by geographical location. In certain regions or industries, VHDL might be more commonly used, while in others, Verilog might be the preferred language.

- **Defense and Aerospace Industry:**

VHDL is often preferred in the defense and aerospace industry. It is the recommended language by the U.S. Department of Defense for electronic design due to its reliability and level of abstraction.

- **Mixed-Signal Design:**

VHDL is considered more suitable for mixed-signal design, where analog and digital components need to be modeled together. Its features for modeling hardware at different abstraction levels make it versatile for mixed-signal applications.

- **Concurrent Signal Assignment:**

VHDL provides concurrent signal assignment, making it easy to describe parallel operations. This feature can be advantageous in certain design scenarios.

Cont.

- **Strong Typing and Abstraction:**

VHDL is known for its strong typing and high level of abstraction. This can be beneficial for system-level modeling and can help catch errors early in the design process.

- **Academic Curricula:**

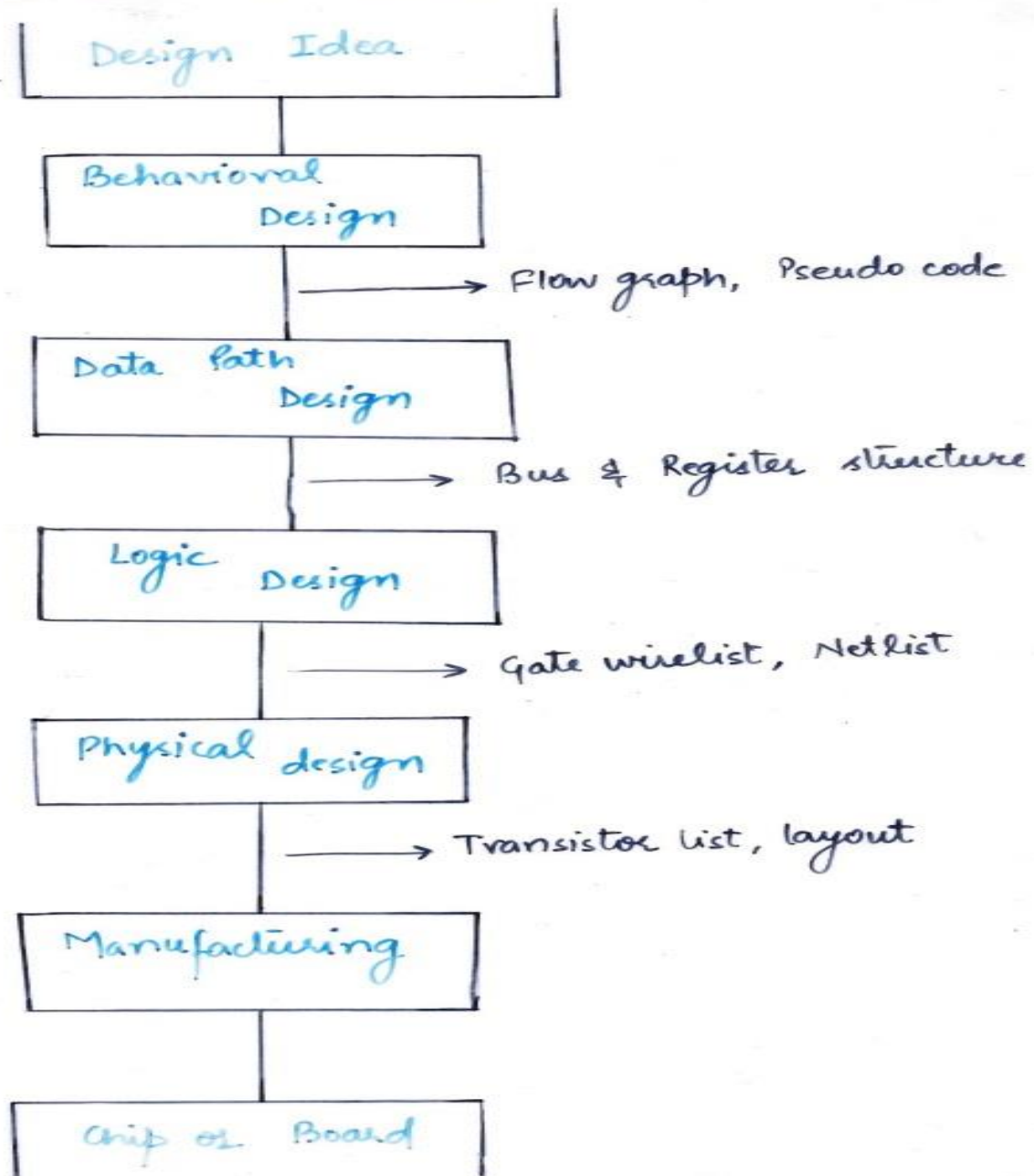
In some academic institutions, VHDL might be the language of choice for teaching digital design and FPGA programming. Students who learn VHDL in their academic programs might continue to use it in their professional careers.

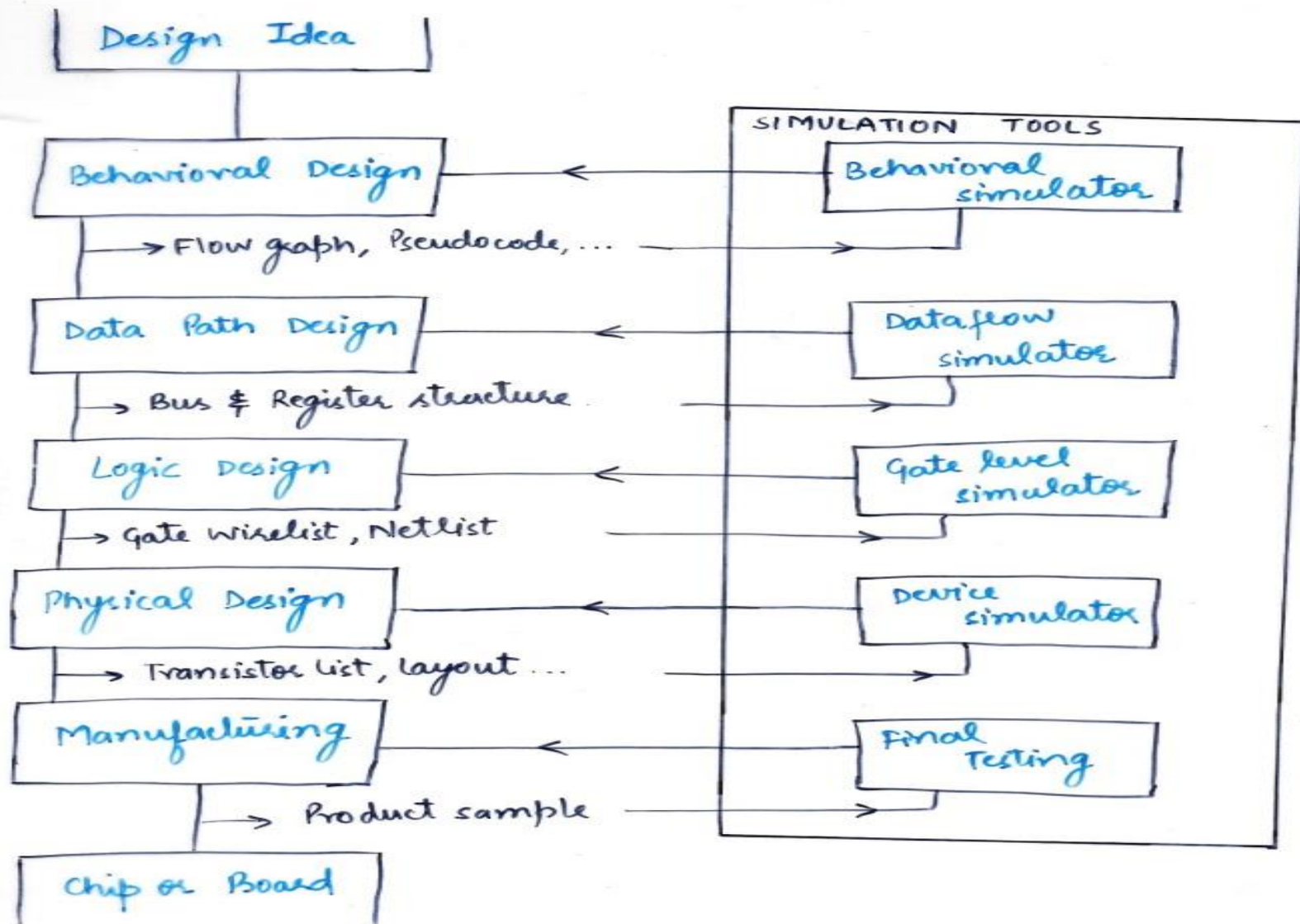
- **Government and Regulatory Standards:**

In some industries, especially those with strict regulatory standards, VHDL may be preferred or even mandated due to its use in certain safety-critical applications.

It's important to note that both VHDL and Verilog have similar capabilities, and proficiency in one language can often be transferred to the other.

A DIGITAL SYSTEM DESIGN PROCESS.





Verifying each design by simulating its output

VHDL: VHSIC HARDWARE DESCRIPTION LANGUAGE.

- Used to model DIGITAL SYSTEM At various levels of Abstraction (from Algorithm to Gate level).
- System can be described hierarchically.
- Language is Amalgamation of
 - Sequential language
 - +
 - Concurrent Language
 - +
 - Netlist language
 - +
 - Timing specifications
 - +
 - Waveform language
- Language has constructs for
 - * concurrent / sequential Behavior with/ without Timing.
 - (allows as) * Modelling as interconnection of component
 - * Test waveform Generation.

VHDL BACKGROUND

2.

- Inhouse HDL were in use.
- design Exchange were impossible.

Department of Defence (DoD) US has required a STANDARD MEDIUM OF EXCHANGE OF DESIGNS.
($\therefore$, Different companies were using diffⁿ HDL's to describe their IC's).

IBM, TEXAS INSTRUMENTS AND INTER METRICS WERE CONTACTED IN 1983. (VHDL Ver 7.2 released in 1985)

IT also has ANSI standards.

1993

IEEE Std 1076-1993.

CAPABILITIES:

3.

→ Exchange medium between chip vendors and CAD Tool users.

→ Communication medium between CAD & CAE Tools.

Schematic
capture

Generates

VHDL
description

CAE

Input → simulation
program

→ supports hierarchy (modeled as set of components)

→ supports flexible design methodology.

TOP DOWN, BOTTOM UP, MIXED

→ Not Technology specific. (support technology-specific features)

→ Various modelling techniques are supported.

ASM, FSM, Boolean equations.

→ Portable as it is IEEE & ANSI standard.

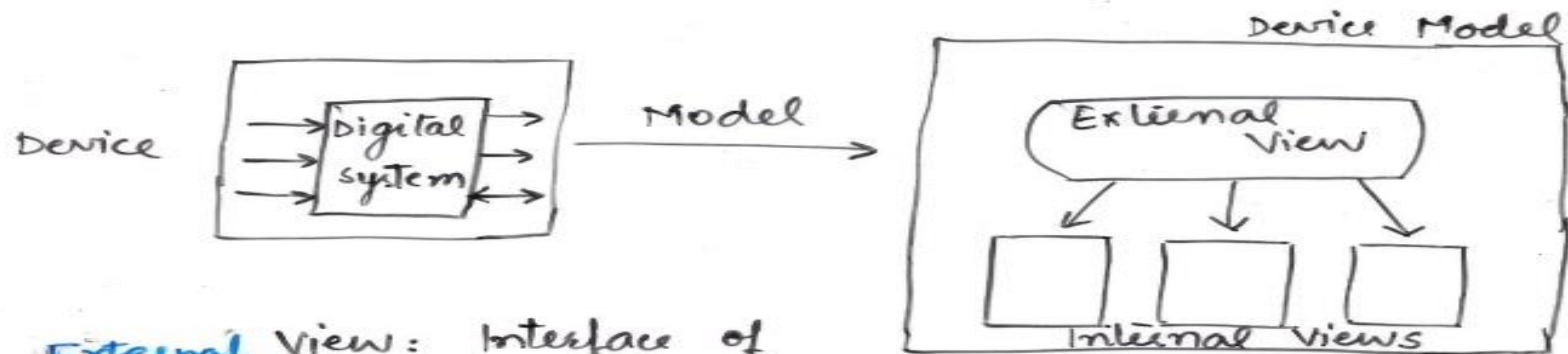
→ wide range of abstraction (behavioural)

to very primitive gate level).

does not support below transistor level.

- Elements that make large scale design modelling easier (function, components, procedures and packages).
- Nominal propagation delays, min-max delays, set up & hold time, timing constraints and spike detection can be described naturally.
- Generic Design
- Back Annotation.
- Capability of defining new datatypes.

HARDWARE ABSTRACTION



External View: Interface of device through which it communicates.

Internal View: Functionality

BASIC TERMINOLOGY

5.

- Hardware Abstraction of system is called Entity.
- Entity X when used within Entity Y becomes component of Y.

Design units are used to describe Entity, types of design units:

- # Entity Declaration (P.D.U.)
- # Architecture Body (S.D.U.)
- # Configuration Declaration (P.D.U.)
- # Package Declaration (P.D.U.)
- # Package Body (S.D.U.)

- P.D.U. can exist on its own.
- S.D.U. can't exist on its own and it is not possible to analyze it before P.D.U.

Entity Modelling

6.

Declaration

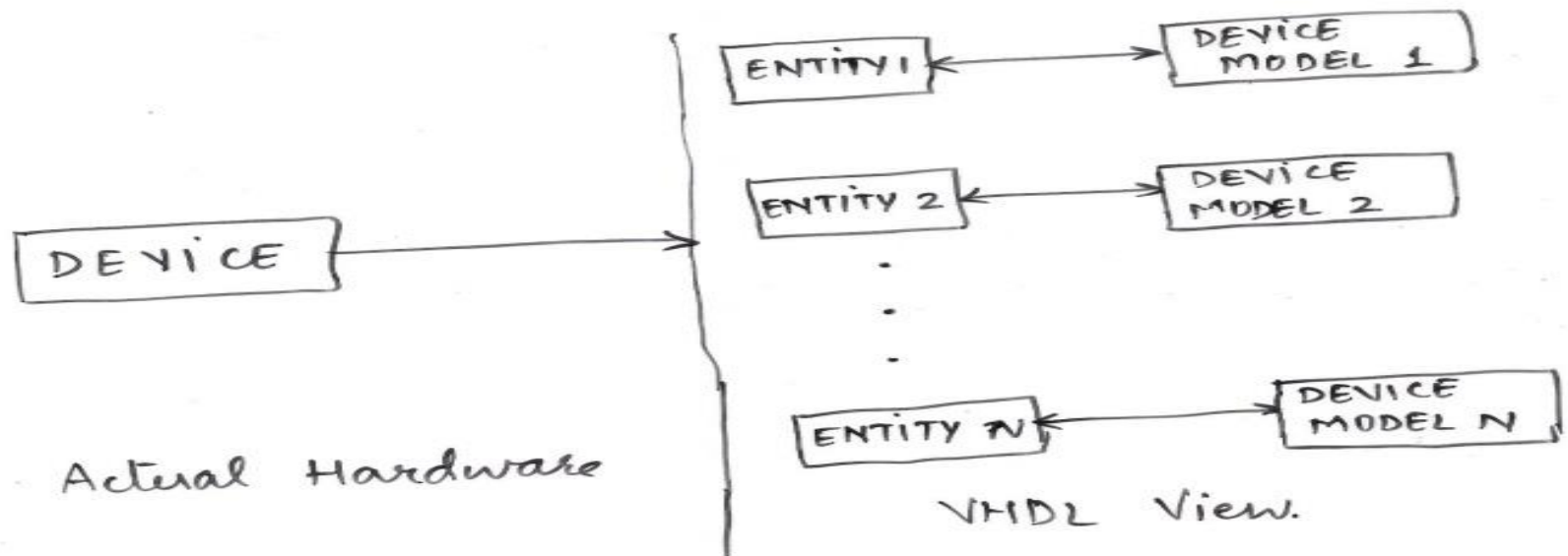
[EXTERNAL VIEW]

+

Architecture Body

[INTERNAL VIEW].

DEVICE TO DEVICE MAPPING IS ONE TO MANY



→ Entity has one external view and one or more internal views

→ Hardware Device may be represented by one or more entities.

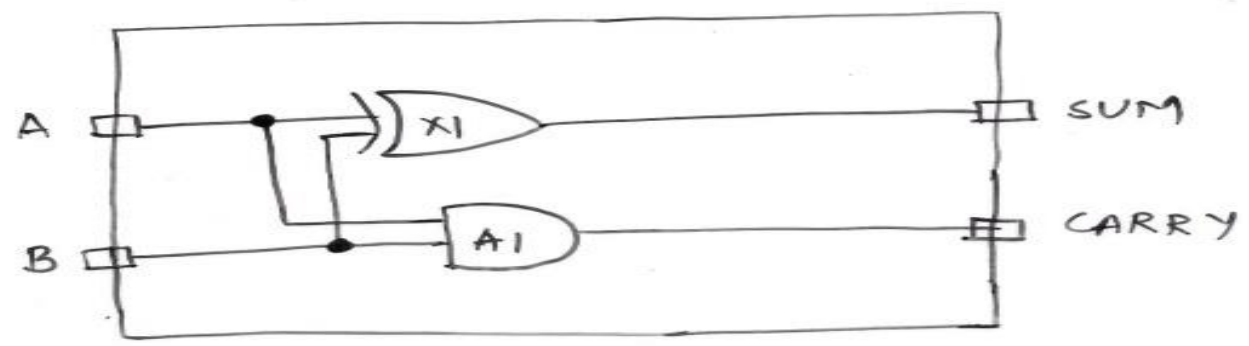
~~REVIS~~

Entity Declaration

- # Specifies name of entity being modelled and list of set of interface ports.
- # Ports are signals with which entity communicates with other models in its external environment.

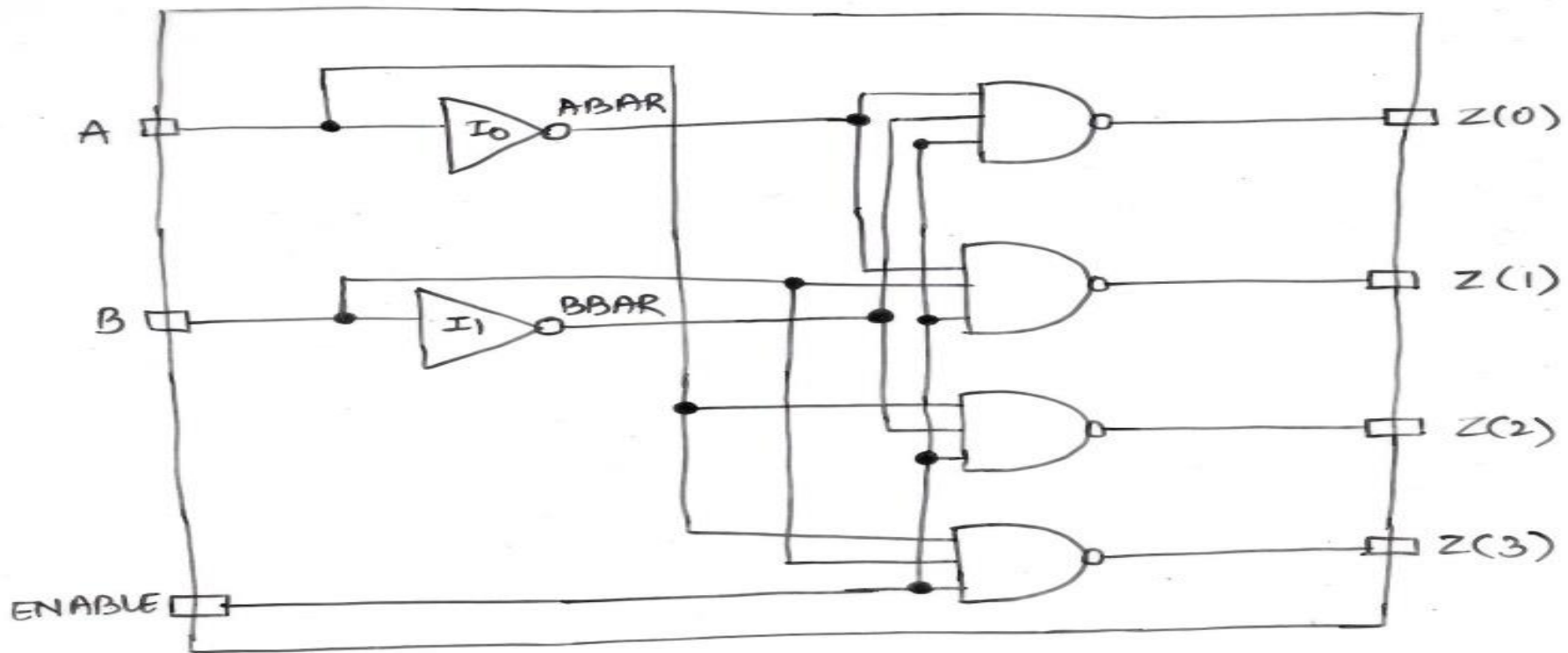
```
entity HALF_ADDER is  
  port (A, B : in BIT;  
        SUM, CARRY: out BIT);  
end HALF_ADDER;
```

· Bit can take value '0' or '1'.



DECODER

18.

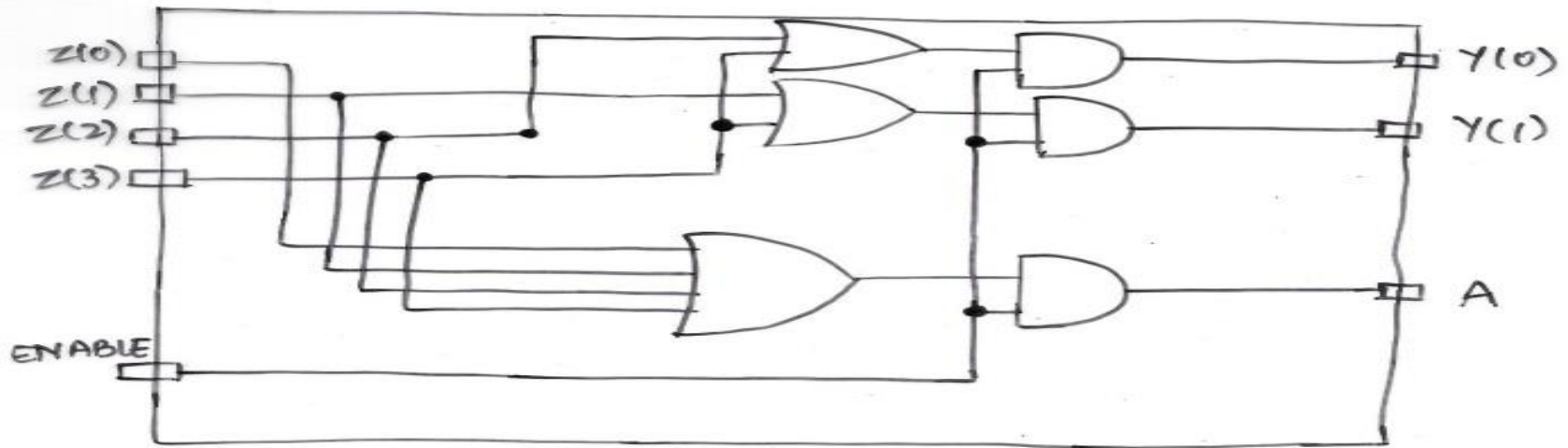


entity DECODER2X4 is
 port (A, B, ENABLE : in BIT;
 Z : out BIT; VECTOR (0 to 3));
 end DECODER2X4;

Bit_vector is predefined constrained array
 type of BIT range "0 to 3".

ENCODER 4X2

9.



Output port A shows whether module is active or not.

entity ENCODER4X2 is

port (z: in BIT_VECTOR (0 to 3);

ENABLE: in BIT;

Y: out BIT_VECTOR (0 to 1);

A: out BIT);

end ENCODER4X2;

ARCHITECTURE BODY

10.

REPRESENTATION STYLES

- As interconnection of components
STRUCTURAL
- As concurrent assignment statements.
DATAFLOW
- As sequential assignment statements
BEHAVIORAL
- Combination of above three.

STRUCTURAL STYLE OF MODELLING

architecture HA_STRUCT of HALF_ADDER is

component XOR2

port (X, Y: in BIT; Z: out BIT);

end component;

component AND2

port (L, M: in BIT; N: out BIT);

end component;

begin

X1: XOR2 portmap (A, B, SUM);

A1: AND2 portmap (A, B, CARRY);

end HA_STRUCT;

Architecture body has 2 parts

- Declarative
- Statement
- XOR2 & AND2 may be predefined or they will later be bound to components in Library.
- X1 & A1 and labels for component instantiation.
- Separate Entity models are/may be required for XOR2 & AND2.

architecture EN-ARC of ENCODER4X2 is
component OR2 is

port (L1, L2: in BIT; L3: out BIT);
end component;

component AND2 is

port (N1, N2: in BIT; N3: out BIT);
end component;

component OR4 is

port (M1, M2, M3, M4 : in BIT;
M5: out BIT);

end component;

signal A1, A2, A3 : BIT;

12.

begin

P0: OR2 portmap (Z(1), Z(3), A1);

P1: OR2 portmap (Z(2), Z(3), A2);

P2: AND2 portmap (A1, ENABLE, Y(1));

P3: ~~AND~~ AND2 portmap (A2, ENABLE, Y(0));

P4: OR4 portmap (Z(1), Z(2), Z(3), Z(4), A3);

P5: AND2 portmap (A3, ENABLE, A);

end EN-ARC;

architecture DEC_STR is DECODER2X4 is

component INV

port (PIN : in BIT; POUT : out BIT);

end component;

component NAND3

port (D0, D1, D2 : in BIT; D2 : out BIT);

end component;

signal ABAR, BBAR; BIT;

begin

V0: INV port map (A, ABAR);

V1: INV port map (B, BBAR);

N0: NAND3 port map (ABAR, BBAR, ENABLE, Z(0));

N1: NAND3 port map (ABAR, B, ENABLE, Z(1));

13

```
N2: NAND3 port map (A, BBAR, ENABLE, Z(2));  
N3: NAND3 port map (A, B, ENABLE, Z(3));  
end DEC_STR;
```

Name of architecture body is DEC_STR and it is associated with entity declaration with the name DECODER 2x4 therefore it inherits the list of interface ports from that entity declaration. Signals ABAR and BBAR represent wires which are used to connect the various components that form decoder. Scope of these signals is restricted to architecture body therefore these signals are not visible outside the architecture body.

POSITIONAL ASSOCIATION IS USED

STRUCTURAL STYLE OF MODELLING DESCRIBES ONLY INTERCONNECTION OF COMPONENTS WITHOUT IMPLYING ANY BEHAVIOUR OF COMPONENT THEMSELVES NOR OF THE ENTITY THAT THEY COLLECTIVELY REPRESENT

DATAFLOW STYLE OF MODELLING

14.

- ENTITY IS EXPRESSED USING CONCURRENT SIGNAL ASSIGNMENT STATEMENTS.
- STRUCTURE OF ENTITY IS NOT SPECIFIED EXPLICITLY BUT CAN BE DEDUCED IMPLICITLY.

architecture HA_CONCURRENT of HALF_ADDER is
begin

SUM <= A XOR B after 8ns;

CARRY <= A and B after 4ns;

end HA_CONCURRENT;

- CONCURRENT SIGNAL ASSIGNMENTS ARE EXECUTED WHEN THERE IS AN EVENT ON ANY SIGNAL on RHS.
- To include time delay 'after' clause is used.

architecture HS_CONCURRENT of HALF_SUBTRACT is
signal BBAR;

begin

DIFF <= A XOR B after 8ns;

BORROW <= A and BBAR;

BBAR <= not B;

end HS_CONCURRENT;

architecture DEC- DATAFLOW of DECODER2X4 is
signal ABAR, BBAR : BIT;

15.

begin

Z(3) <= not (A and B and ENABLE); -- stmt 1

Z(0) <= not (ABAR and BBAR and ENABLE); -- stmt 2

BBAR <= not B; -- stmt 3

Z(2) <= not (A and BBAR and ENABLE); -- stmt 4

ABAR <= not A; -- stmt 5

Z(1) <= not (ABAR and B and ENABLE); -- stmt 6

end DEC- DATAFLOW;

- If 'after' clause is not used then a default delay of 0ns is assumed known as DELTA DELAY.

If there is an event at time T on A stmts 1, 4 & 5 will be triggered. LHS will get value at $T + \Delta$. 2, 5 & 6 will then be triggered at $T + 2\Delta$.

Z(0) and Z(1) will get value.

Behavioral style of Modeling

- Statements are executed sequentially in a specified order.
- All statements are written inside a process statement.

Syntax for Process statement.

```

Process ( sensitivity - list )
begin
    ==
end process

```

eg1: 2X4 Decoder

Architecture dec-seq of decoder24 is

begin

process (A, B, Enable)

Variable Abar, Bbar : bit;

begin begin

Abar := not A;

Bbar := not B;

if ENABLE = '1' Then

Z(3) <= not (A and B);

Z(2) <= not (A and Bbar);

Z(1) <= not (Abar and B);

Z(0) <= not (Abar and Bbar);

else
Z <= "1111";
end if;
end process;
end dec-seq;

eg 2: Level sensitive DFF

Entity LS_DFF is
 port (Q: out BIT; D, CLK: in BIT);
 end LS_DFF;

Architecture DFF_beh of LS_DFF is

```
begin
  process (D, CLK)
    begin
      if CLK = '1' then
        Q <= D;
      end if;
    end process;
  end DFF_beh;
```

Mixed style of Modelling:

* To mix Three modelling styles in a single architecture body i.e. we could use:

- Component instantiation statements
- Concurrent Assignment statements
- Process statements.

entity Full-adder is

port (A, B, Cin : in Bit; sum, carry : out Bit);

end Full-adder.

Architecture FA-mixed of Full-adder is

component XOR2

port (P1, P2 : in Bit; Pz : out Bit); -- structure

end component;

signal S1: Bit;

begin

X1: XOR portmap (A, B, S1);

process (A, B, Cin)

variable T1, T2, T3: Bit;

begin

T1 := A and B;

T2 := B and Cin;

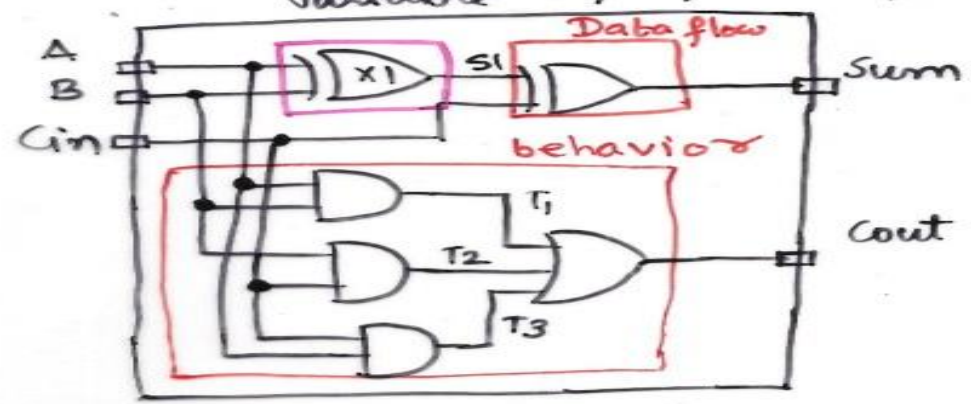
T3 := Cin and A;

Cout <= T1 or T2 or T3;

end process;

Sum <= S1 xor Cin;

end FA-mixed -- data-flow



Configuration Declaration:

- Used to select one of possibly many architecture bodies
- To bind components used in the architecture body.

Package Declaration:

- Used to store a set of common declarations, such as components, types, procedures and functions.
- These declarations can be imported into other design units using **USE** clause.

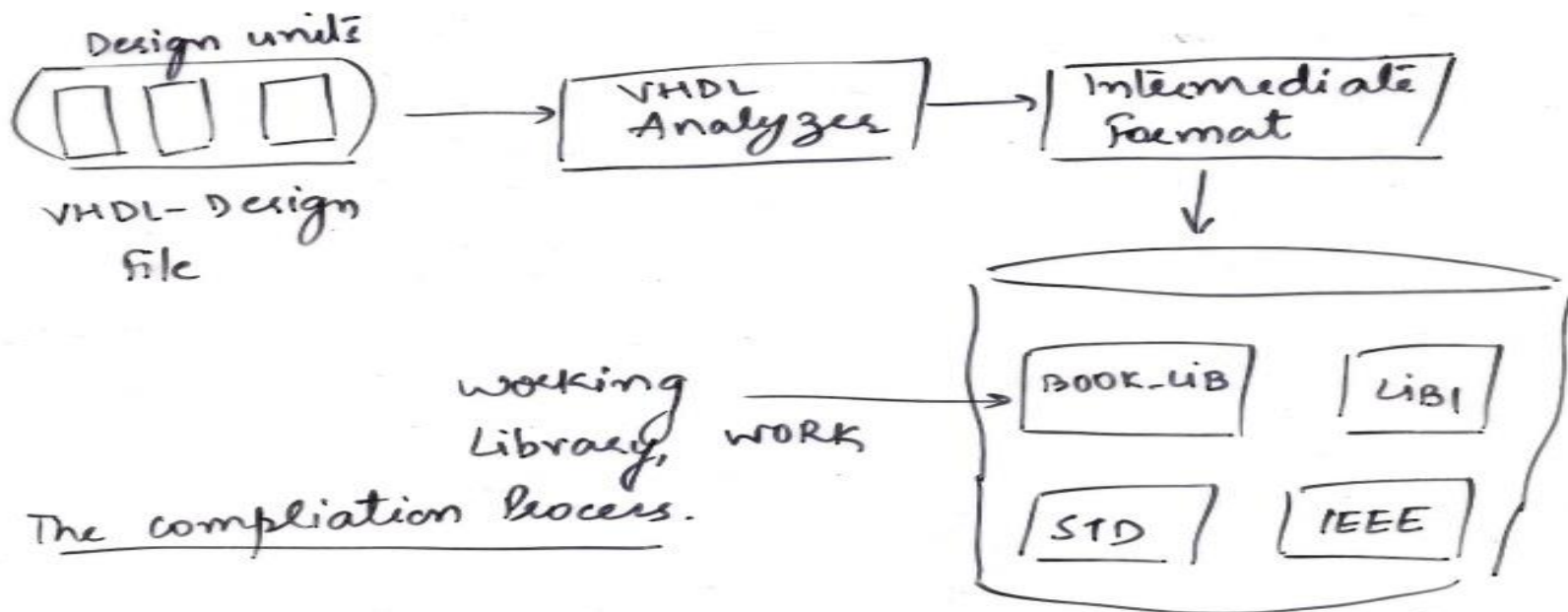
Package Body:

- A package body is always associated with a package declaration.
- A package body is used to store the definitions of functions and procedures that were declared in the corresponding package declaration.

Model Analysis

VHDL design file is validated using

- Analyzer (do Analysis)
- simulator



The compilation process.

- One particular library designated as working library with logical name WORK. (compiles descriptions in this file).
- Items compiled in one design library can be imported into design units compiled in different design library by using library and use clauses;
- One predefined design library, STD has 2 packages:
 - STANDARD (declarations for all data types (BIT, TIME etc.))
 - TEXTIO (procedures & functions).

Simulation:

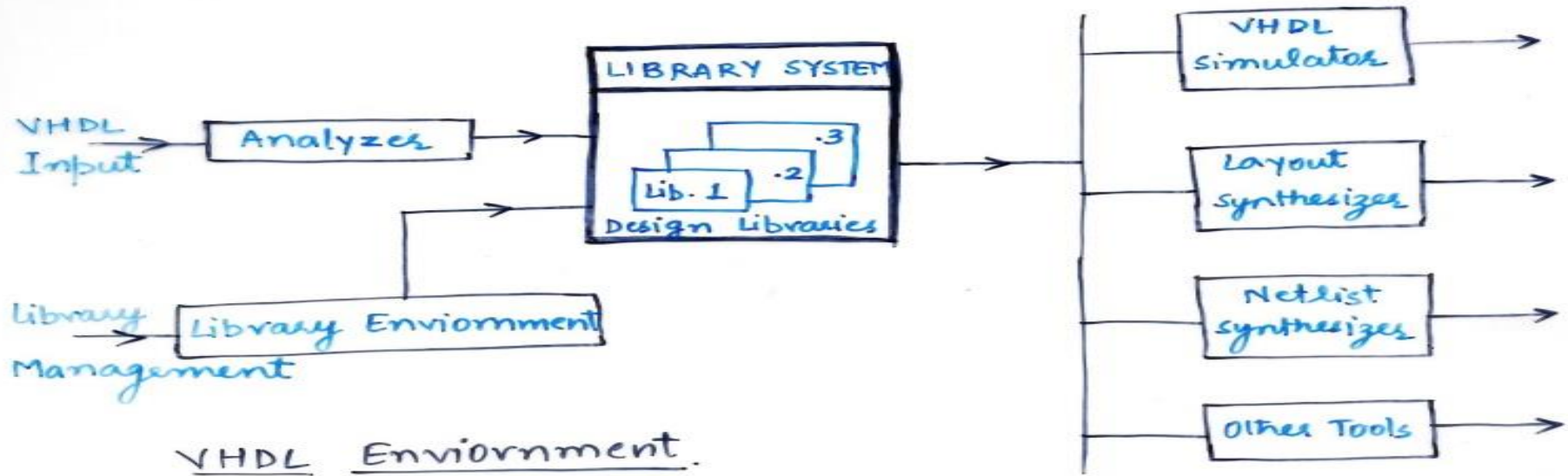
Preceding the actual simulation are two major steps:

1. Elaboration phase:
 - ① Hierarchy of Entity linked
 - ② Storage Allocation & assign initial values
2. Initialization Phase:
 - ① Effective values for all explicitly declared signals, are computed
 - ② Implicit signals are assigned values.

The VHDL language:

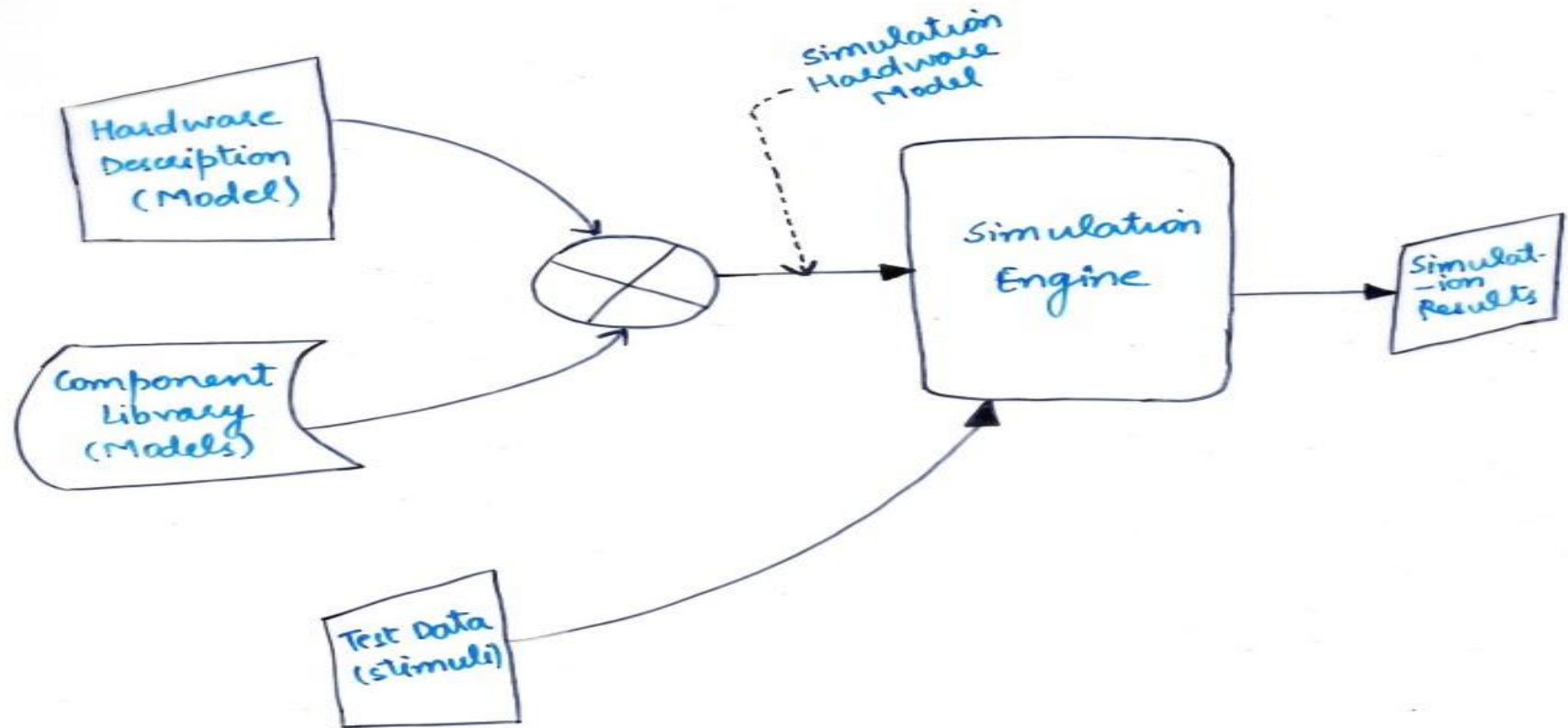
- VHDL satisfies all requirements of 1983 DoD requirements document.
- VHDL is an HDL with strong emphasis on concurrency.
 - supports hierarchical description of hardware from system to gate to even switch level.
 - strong support at all levels for timing specification and violation detection.
- A VHDL design entry is defined as an entity declaration and an associated architecture body.
- The VHDL language groups subprograms or design entities by use of packages.
 - For customizing generic descriptions of design entities, configurations are used.
- VHDL also supports libraries and contain constructs for accessing packages, design entities or configuration from various libraries.

- A typical VHDL design environment is depicted. 202



- Analyzer program converts a VHDL description into a intermediate form and places it into design library.
 → Analyzer is responsible for lexical analysis and syntax check.
- operations on design library such as creating new libraries, deleting the old libraries or deleting packages or design entities from a library, are done through design environment.
 → VHDL based tools used the intermediate form from design library e.g. VHDL simulator which simulates a design entry from the design library and produces a simulation report.

HARDWARE SIMULATION



Simulation:

Simulation is an act of exercising a model of an actual components for analyzing its conduct under a given set of conditions and/or stimuli.

- simulation engine may be capable of producing detailed timing and analog voltages or it may be able to produce high level function information.

Objects and Data types in VHDL

VHDL defines several types of **objects**. These include **constants**, **variables**, **signals** and **files**.

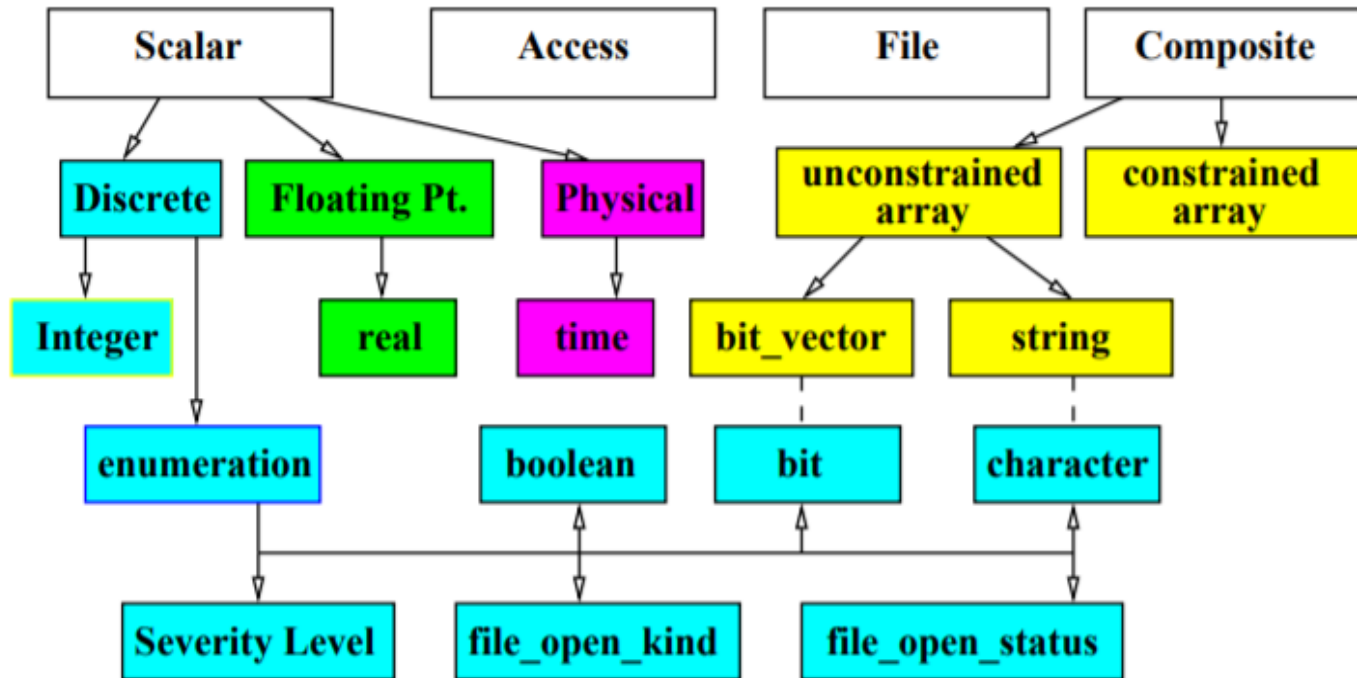
The types of values which can be assigned to these objects are called *data types*.

Same *data* types may be assigned to different *object* types. For example, a **constant**, a **variable** and a **signal** can all have values which are of *data type* **BIT**.

Declarations of objects include their *object* type as well as the *data* type of values that they can acquire.

For example **signal** Enable: **BIT**;

Data Types



Enumeration type

VHDL *enumeration* types allow us to define a set of values that a variable of this type can acquire. For example, we can define a data type by the following declaration:

```
type instr is (add, sub, adc, sbb, rotl, rotr);
```

Now a variable or a signal defined to be of type instr can only be assigned values enumerated above – that is: add, sub, adc, sbb, rotl and rotr.

In actual implementation, these values may may be mapped to a 3 bit value. However, an attempt to assign, say, '010' to a variable of type instr will result in an error. Only the enumerated values can be assigned to a variable of this type.

Predefined Enumeration type

A few enumeration types are pre-defined in the language.

These are:

type bit **is** ('0', '1');

type boolean **is** (false, true);

type severity_level **is** (note, warning, error, failure);

type file_open_kind **is** (read_mode, write_mode, append_mode);

type file_open_status **is**
 (open_ok, status_error, name_error, mode_error);

In addition to these, the character type enumerates all the ASCII characters.

Types and subtypes

A signal type defined in the IEEE Library is `std_logic`. This is a signal which can take one of 9 possible values. It is defined by:

```
type std_logic is ('U', 'X', '0', '1', 'Z', 'W', 'L', 'H', '-');
```

A subtype of this kind of signal can be defined, which can take the four values 'X', '0', '1', and 'Z' only.

This can be defined to be a subtype of `std_logic`

```
subtype fourval_logic is std_logic range 'X' to 'Z';
```

Similarly, we may want to constrain some integers to a limited range of values. This can be done by defining a new type:

```
subtype bitnum is integer range 31 downto 0;
```


Physical type

Objects which are declared to be of Physical type, carry a value as well as a unit. These are used to represent physical quantities such as time, resistance and capacitance.

The Physical type defines a basic unit for the quantity and may define other units which are multiples of this unit.

Time is the only Physical type, which is pre-defined in the language. The user may define other Physical types.

Predefined Physical type: Time

```
type time is range 0 to ...  
  units  
    fs;  
    ps = 1000 fs;  
    ns = 1000 ps;  
    us = 1000 ns;  
    ms = 1000 us;  
    sec = 1000 ms;  
    min = 60 sec;  
    hr = 60 min;  
  end units time;
```

The user may define other physical types as required.

User defined Physical Type

As an example of user defined Physical types, we can define the resistance type.

```
type resistance is range 0 to 1E9  
  units  
    ohm;  
    kohm = 1000 ohm;  
    Mohm = 1000 kohm;  
end units resistance;
```

Composite Data Type

Composite data types are collections of scalar types.

VHDL recognizes records and arrays as composite data types.

Records are like structures in C.

Arrays are indexed collections of scalar types. The index must be a discrete scalar type.

Arrays may be one-dimensional or multi dimensional.

Arrays

Arrays can be constrained or unconstrained.

- In constrained arrays, the type definition itself places bounds on index values. For example:

type byte **is array** (7 downto 0) **of** bit;
type rotmatrix **is array** (1 to 3, 1 to 3) **of** real;
- In unconstrained arrays, no bounds are placed on index values. Bounds are established at the time of declaration.

type bus **is array** (natural range <>) **of** bit;

The declaration could be:

signal addr_bus: bus(15 downto 0);

signal data_bus: bus(7 downto 0);

Shift operator

- Sll (shift left logical)
- Srl (shift right logical)
- Sla(shift left arithmetic)
- Sra(shift right arithmetic)
- Rol (rotate left)
- Ror(rotate right)

Shift operator continues

- “1001010” sll2 “0101000”
- “1001010” sla2 0101000 rightmost bit filled with 0
- ““1001010” sra3 filled with 1 the leftmost bit
- “1001010” rol2
- “1001010” ror3
- “1001010” srl-5 perform sll5
- “1001010” sla-2 sra2
- “1001010” sra-4 sla4
- “1001010” rol-1 ror1
- “1001010” sll-5 srl5

Adding operator and multiplying operator

- “&”
- ‘c’ & ‘a’ & ‘t’ “cat”
- “*”, “/” mod , rem
- $A \text{ rem } b = a - (a/b) * b$
- $A \text{ mod } b = a - b * n$

Data Flow Modeling:

Concurrent signal Assignment statement:

Architecture OR2 of OR2 is

begin

$Z \leftarrow A \text{ or } B$ after 9ns;

end OR2;



-- Data flow of OR gate.

- When event occurs on A or B, expression on RHS evaluated and its value is scheduled to appear on signal Z after 9ns.
- Architecture contains any number of concurrent signal assignment.

Concurrent Vs sequential signal Assignment:

- Concurrent signal assignment statements are event-triggered i.e. executed whenever event on a signal that appears in its expression while sequential are not event-triggered and are executed in sequence in relation to other sequential statements.

Architecture seq-sig-Asn of Frag is

begin

process(B)

begin

$A \leftarrow B;$

$Z \leftarrow A;$

end process;

end;

Architecture con-Asn of Frag:

begin

$A \leftarrow B;$

$Z \leftarrow A;$

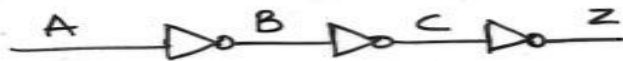
end;

- In seq-sig-Asg, two signal assignments are sequential. DM-2
Whenever signal B has an event, first signal assignment is executed and then the second, both in zero time.
sigA is scheduled to get its ^{new} value of B at $T+\Delta$,
and Z gets the old value of A at $T+\Delta$.

- In con-sig-Asg, when event occurs on B, sig A gets the value of B after delay, i.e. $T+\Delta$.
New value of A assigned to Z after $T+2\Delta$.

Delta Delay Revisited:

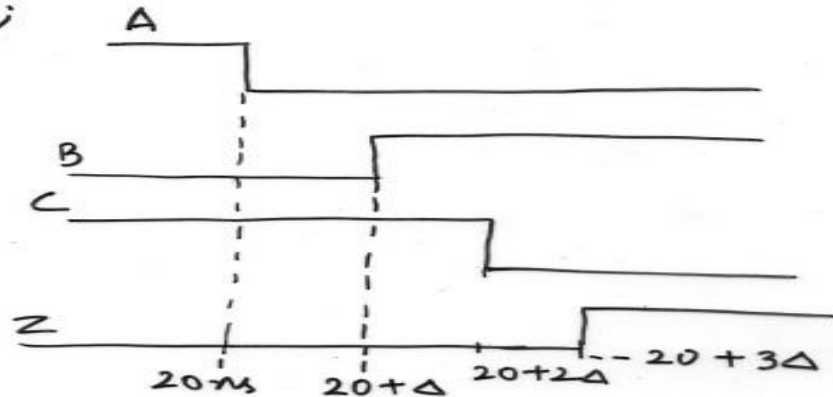
- It is an infinitesimally small amount of delay.
- does not cause real simulation time to change.



entity fast_inverter is
port (A: in bit; Z: out bit);
end;

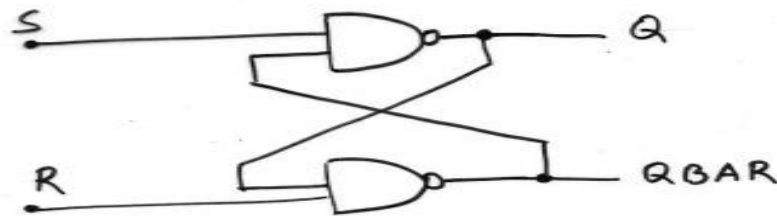
Architecture Delta-delay of fast_inverter is
signal B, C: bit;

```
begin
  Z <= not C;
  C <= not B;
  B <= not A;
end;
```



ORDERING OF EVENTS IN RS LATCH:

DM-3



Entity RS_Latch is

port (R, S : in BIT; Q : Buffer BIT;
QBAR : buffer BIT);

end RS_Latch;

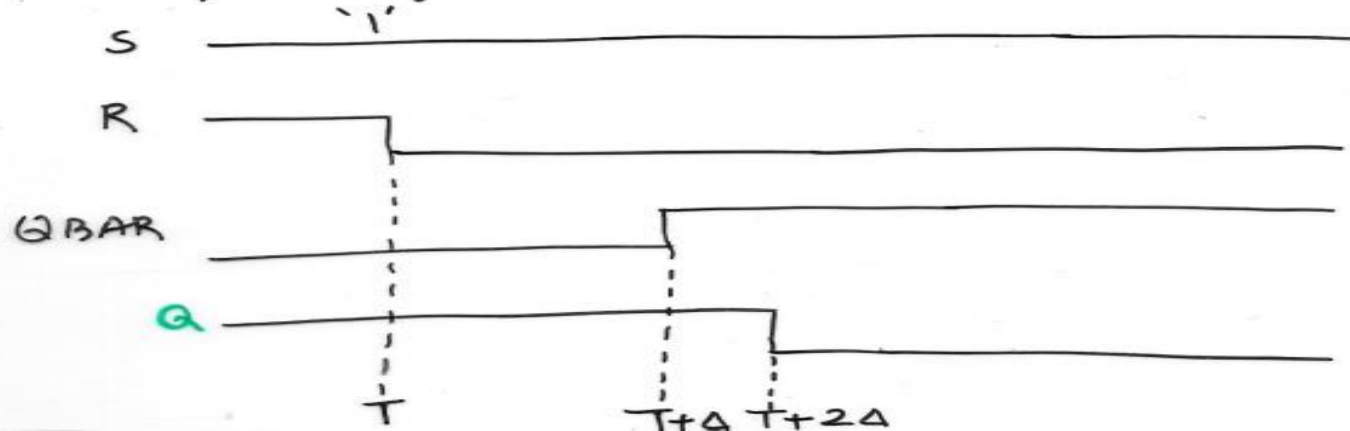
Architecture con of RS_Latch is
begin

QBAR $\leftarrow$ R nand Q;

Q $\leftarrow$ S nand QBAR;

end con;

sequence of events:

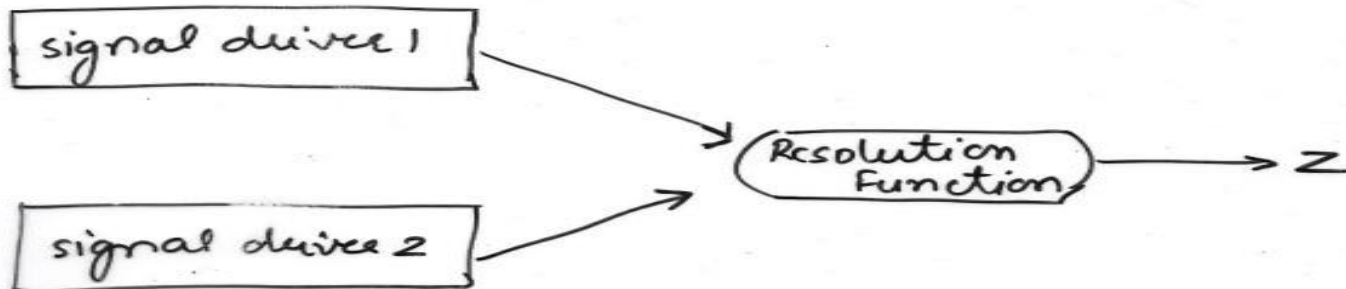


Multiple Drivers:

entity two-de-eg is
 port (A, B, C : in BIT; Z : out BIT);
 end two-de-eg;
 Architecture arch of two-de-eg is
 begin
 $Z \leq A \text{ and } B;$
 $Z \leq \text{not } C;$
 end arch;

How is the value of Z determined?

By using a user defined fn "RESOLUTION FUNCTION"



eg: Architecture No. entity of Dummy is begin

begin

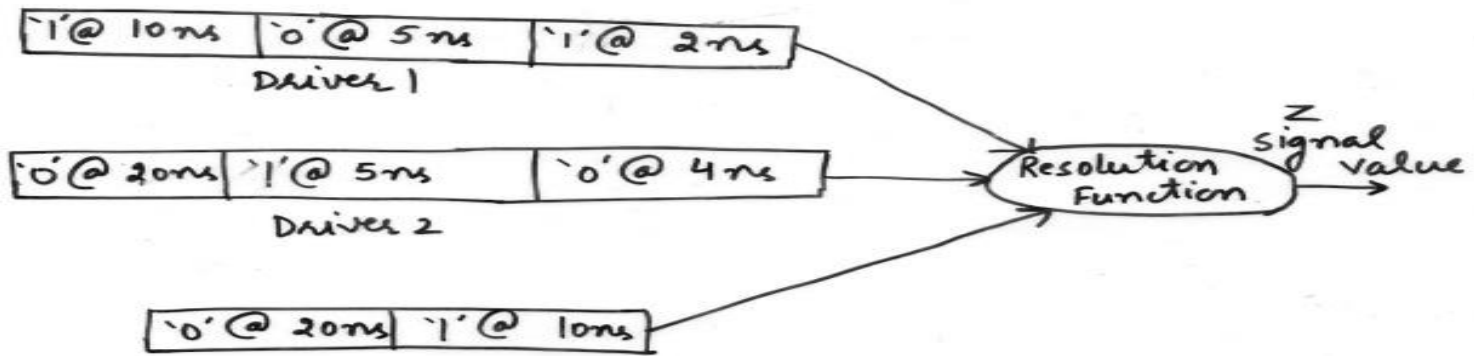
$z \leq '1'$ after 2ns, $'0'$ after 5ns, $'1'$ after 10ns;

$z \leq$ '0' after 4ns, '1' after 5ns, '0' after 20ns;

$z \leftarrow$ '1' after 10ns, '0' after 20ns;

end NO_entity;

- has three drivers of Z . Each driver has sequence of transactions.



- Resolution function is user written & may perform any function.
- A signal with more than one driver must have resolution function associated with it, ^{otherwise} it is an error. Such a signal is called Resolved signal.

```
port (A, B, C : in BIT;  Z : out WIRED-OR BIT);
```

↓
Function

- Whenever an event occurs on any source of a DM-6 resolved signal, the resolution fn associated with it is called with value of all its drivers. The return value from the resolution function becomes the value of the resolved signal.

Function WIRED_OR (INPUTS: BIT_VECTOR) return BIT is
begin

For J in INPUTS' RANGE loop

if INPUTS(J) = '1' then
return '1';

endif;

end loop;

return '0';

end WIRED_OR;

- A signal is recognized as a resolution function if it is associated with a signal in the signal declaration or a subtype declaration.

Conditional signal Assignment statement - DM-7

Syntax:

(like if statement)

target signal <= [waveform-elements when condition else
[waveform-elements when condition else]
...
waveform-elements [when condition];

eg:

Z <= IN0 after 10ns when S0 = '0' and S1 = '0' else
IN1 after 10ns when S0 = '1' and S1 = '1' else
IN2 after 10ns when S0 = '0' and S1 = '1' else
IN3 after 10ns;

Event occurs on IN0, IN1, IN2, IN3. & S0 and S1.

Selected signal Assignment statement - (like case statement)

Syntax:

With expression select
target-signal <= waveform-element when choices,
waveform-element when choices,
=
waveform-element when choices;

eg- type OP is (ADD, SUB, MUL, DIV)

signal OP_CODE : OP;

with OP_CODE select

$Z \leq A + B$ when ADD,

$A - B$ when SUB,

$A \times B$ when MUL,

A / B when DIV;

- Whenever event occurs on the signal in the select expression (OP_CODE) or on any signal used in any of the waveform elements (A, B).
- Note all the choices are not evaluated in sequence.
- All choices must be covered exactly once.
- Values not covered explicitly may be covered by "others".

Unaffected value: (like null statement)

- causes no change to the driver for the target signal.

eg:

type STATE_TYPE is (Reset, Apply, waits, Hold, Receive);

signal NEXT_STATE : STATE_TYPE;

...

with NEXT_STATE select

ZRX <= B"0001" when APPLY,
 B"0010" when WAITS,
 B"0100" when RESET,
 unaffected when others;

Unaffected does not cause any change to the driver i.e. ZRX retains its old value.

eg- MARK_FLAG <= BKDET after 5ns when STROBE = '0'
 else unaffected;

BLOCK STATEMENT:

used for three major purposes:

1. To disable signal drivers using guards.
2. To limit scope of declarations, including signal declarations.
3. To represent a portion of design.

Syntax: → It is necessary.

block-label : block [guard-expression] [is]
describes interface of block statement to its environment. [block header] → must be boolean type.
begin [block-declarations] -- visible only in block
 concurrent statements -- Any no. of statements can appear.
 end block [block-label]; → optional

- If guard-expression appears in the block statement, there is implicit signal GUARD of type BOOLEAN declared within the block.

- $GUARD^R$ signal can be used to enable or disable the drivers (of signals).

eg: B1: BLOCK (STROBE = '1')
begin

 Z <= guarded not A;
end block B1;

→ optional (when value of guard signal is true, value of expression "not A" assigned to Z).

- When guard signal is false, events on A do not affect the value of signal Z. i.e. drivers to Z for this is disabled.

- Block statements are very useful in modelling hardware elements that trigger on certain events eg: FF's & clocked logic.

- signal guard implicitly declared, can be used explicitly within a block statement.

eg: B2: BLOCK (CLEAR = '0' and PRESET = '1')

statement executed any time event occurs on guard signal. begin { Q <= '1' ^{when} after not GUARD else '0'; } -- not guarded assignment end BLOCK B2;

- Drivers to Q never disabled.

Library ieee;

Use ieee.std_logic_1164.all;

entity e_block is

port (a, b : in bit;

p : out bit);

end e_block;

architecture a_test of e_block is
begin

b1: block (a = '1')

begin

p <= guarded not b;

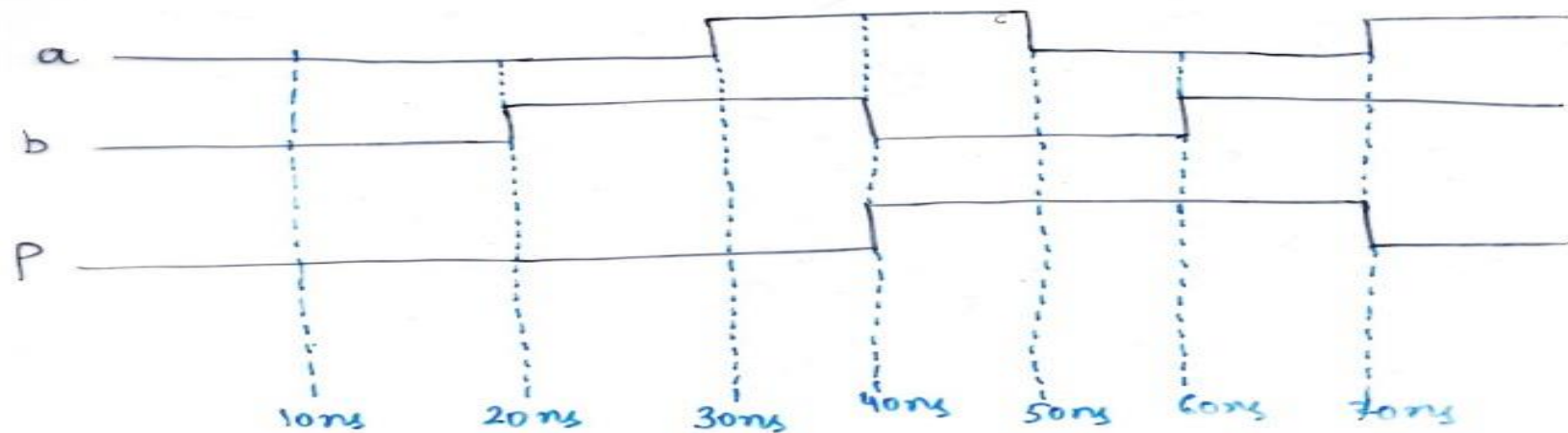
end block;

end a_test;

Implicitly:

Guard a : boolean;

Guard <= guard_expression.



- Possible to explicitly declare a signal called GUARD, DM-11 define an expression for it and then use it within a guarded assignment.

eg: B3: Block

signal GUARD: BOOLEAN;

begin

GUARD <= CLEAR = '0' and PRESET = '1';

Q <= GUARDED DIN;

end block B3;

Concurrent Assertion statement:

- same syntax as in sequential Assertion statement.
- Whenever an event occurs on a signal in the boolean expression of the assertion statement, the statement is executed.

eg: Library IEEE;

Use IEEE.STD_LOGIC_1164.all;

entity SR is

port (S, R: in bit; Q, NOTQ: out bit);

end SR;

Architecture SR_assert of SR is

begin

Assert not (S = '0' and R = '0') -- check R & S are never simultaneously zero.

report "Not valid inputs: R and S are both low."

severity ERROR;

end SR_assert.

```
entity e1 is
  port (D : inout bit);
end e1;
```

```
architecture a1 of e1 is
  signal A, B, C, E, F : bit;
begin
```

```
  C <= A;
  A <= B or D;
```

```
  P1: process (A)
  begin
```

```
    B <= A;
```

```
  end process P1;
```

```
  P2: process
  begin
```

```
    wait until A <= '1';
```

```
    wait for 0ns;
```

```
    E <= B;
```

```
    D <= '0';
```

```
    F <= E;
```

```
  end process P2;
```

```
end a1;
```

B1: Block (guard-expression)

begin

process

begin

if GUARD then

guard-sig <= expression1 after time1;

else

guard-sig <= null;

end if;

wait on signals-in-expression1, GUARD;

end process;

Process

begin

if GUARD then

unguard-sig <= expression2 after time2;

end if;

wait on signals-in-expression2, GUARD;

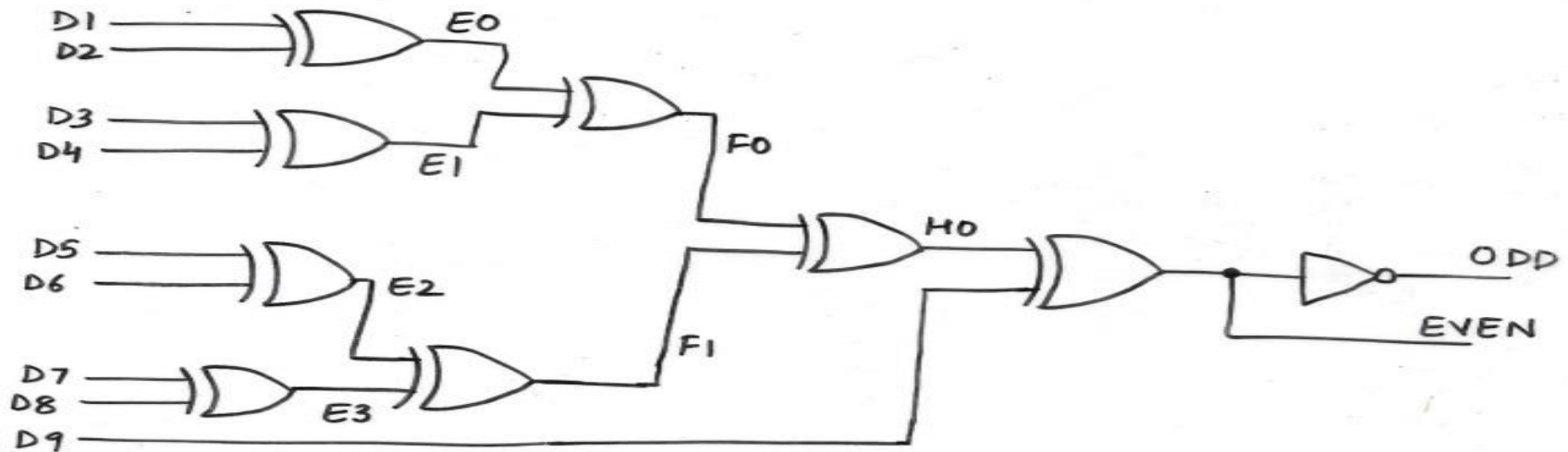
end process;

end block B1;

STRUCTURAL MODELING

An entity is modeled as a set of components connected by signals i.e. as a net list.

eg: 9^{bit} parity Generator



Library IEEE;

Use IEEE.STD_logic_1164.all;

entity Parity-9-bit is

port (D: in STD_logic_vector (9 downto 1);

ODD: out STD_logic;

EVEN: inout STD_logic);

end parity-9-bit;

Architecture parity-str of parity-9-bit is
component XOR2

port (A, B : in STD_logic ; Z: out STD_logic);

end component;

component IN2

port (A: in STD_logic ; Z: out STD_logic);

end component;

signal E0, E1, E2, E3, F0, F1, H0: STD_logic;

begin

XE0: XOR2 port map (D(1), D(2), E0);

XE1: XOR2 port map (D(3), D(4), E1);

XE2: XOR2 port map (D(5), D(6), E2);

XE3: XOR2 port map (D(7), D(8), E3);

XE4: XOR2 port map (E0, E1, F0);

XE5: XOR2 port map (E2, E3, F1);

XE6: XOR2 port map (F0, F1, H0);

XE7: XOR2 port map (H0, D9, EVEN);

INI: INV2 port map (EVEN, ODD);

end parity-str;

COMPONENT DECLARATION

SM-3

- declares the name and the interface of a component
- Interface specifies the mode and the type of ports.

Syntax:

Component component-name [is]

E port (list-of-interface-ports)

specifies the name, mode and type for each port of component.

end component [component-name];

- component-name may or may not refer to the name of an entity already existing in the Library.

If it does not, it must be explicitly bound to an entity; otherwise the model cannot be simulated.

[EN CONFIGURATIONS]. Binding information is there.

- Component declarations appear in the declarations part of an architecture body.

Alternatively, they may also appear in a package declaration. Item declared in this package can be made visible within any architecture body by use Library & Use clause.

eg: package comp-list is
 component and2
 port (X, Y: in BIT; Z: out BIT);
 end component;
 end comp-list;

Above package is compiled in des-lib.

How to include it?

```

Library des-lib;
Use des-lib. comp-list. all;
Architecture
begin
  ...
end;
```

} → can use the component
 AND2 / No need for specifying
 component declarations here.

COMPONENT INSTANTIATION

- A component instantiation statement defines a sub-component of an entity in which it appears.
- It associates the signals in the entity with the ports of that subcomponent.

Format

component-label: component-name [port map (association-list)]

- Association-list associates signals in the entity called Actuals, with the ports of a component called Formals.

Ways to perform the Association of Formals with

SM-5

Actuals:

- Positional Association.
- Named Association.

Positional Association:

- An association list is of the form
actual₁, actual₂, ..., actual_n.
- Each actual in the component instantiation
is mapped by position with each part in the
component declaration.

eg: -- component declaration
component NAND2
 part (A, B: in STD-logic; z: out STD-logic);
end component;
N1: NAND2 part map (s1, s2, s3);

Explanation: In positional Association,
signal s1 is associated with part A of NAND2.
signal s2 is associated with part B of NAND2.
signal s3 is associated with part z of NAND2.

- Ordering of actuals is important

- If a port in a component instantiation is SM-6 not connected to any signal, the keyword **open** can be used to signify that the port is not connected.

eg:

N3: NAND2 port map (S1, open, S3);

Note: (Actual) (declared in entity).
An input port may be left open if its declaration specifies an initial value.

Named Association:

Here, the association list is of the form:

formal1 $\Rightarrow$ actual1, formal2 $\Rightarrow$ actual2 --- so on.

eg:

N1: NAND2 port map (A $\Rightarrow$ S1, B $\Rightarrow$ S2, C $\Rightarrow$ S3);

- Ordering of Associations is not important.
- Type & mode of ports associated must be same.

- Since, a locally declared signal is considered to be both readable & writable such a signal may be associated with a formal of any mode.

- Vectors
- Slices
- Array elements.

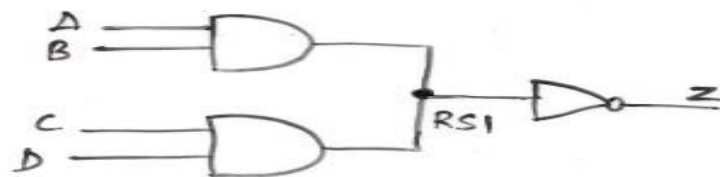
eg:- component child6
 port (inp: in Bit_vector (0 to 4);
 outp: out Bit_vector (3 downto 0);
 end component;
 signal &B: Bit_vector (4 downto 0);
 signal Resut : Bit;
 signal set : Bit_vector (0 to 3);

inst6: child_pos map (inp(0 to 2) $\Rightarrow$ QB(4 downto 2), inp(3) $\Rightarrow$ Reset, inp(4) $\Rightarrow$ QB(0), outp $\Rightarrow$ set);

slices $\leftarrow$ $\downarrow$ Array elements $\downarrow$ Vectors

Resolving signal values:

- If outputs of two components drive a common signal value of signal must be resolved using resolution fⁿ



Entity Driving-signal is
 port (A, B, C, D: in Bit; Z: out Bit);

end driving-signal;

Use work. RF-pack. Pull-up

(Pull up is name of fⁿ defined in package RF-pack).

Architecture Resolved of Driving-signal is

signal RSI: pull-up Bit;

component AND2

port (IN1, IN2: in Bit; Out1: out Bit);

end component;

component INV

port (X: in Bit; Y: out Bit);

end component;

begin

A1: AND2 port map (A, B, RSI);

A2: AND2 port map (C, D, RSI);

I1: INV port map (RSI, Z);

end Resolved;

Values passed through the resolution function before the value is assigned to RSI.

- Each, out, inout and buffer port of a component creates a driver for the signal with which it is associated.

BEHAVIORAL MODELING

BM-1

Entity: An entity declaration describes the external interface of the digital system.

1. In: can only be read.
2. Out: can only be updated.
3. InOut: can be read & updated.
4. Buffer: same, but has only one source & can be connected only to another buffer.

Process statement:

[Process Label:] process (sensitivity list)

[process-item declaration]

begin

sequential statement: these are →

Variable assignment statement.

wait statement.

if statement.

Case statement.

Loop statement.

Null statement.

exit statement.

Next statement.
 Assertion statement.
 Report statement
 end process [process-Label];

e.g.

```
Process (A)
Variable EVENTS_ON_A : INTEGER := -1;
begin
EVENTS_ON_A := EVENTS_ON_A + 1;
end process;
```

Architecture Body:

Variable Assignment statement:

Variable can be declared & used inside a process statement.

Variable := expression

A variable can be declared outside a process. such a variable can be read and updated by more than one process and are called shared variables.

Signal Assignment statement:

signal-object $\leftarrow$ expression [after delay-value];

$z \leftarrow '0'$ after 10 ns;

- * can appear within a process or outside a process.

COUNTER $\leftarrow$ COUNTER + "0010";

Delta delay: Δ , allows for ordering of events.

Wait statement:

It provides an alternate way to suspend the execution of a process.

Process

Variable T1, T2 : BIT;

begin

T1 := A and B;

T2 := C and D;

T1 := T1 or T2;

z $\leftarrow$ Not T1;

wait on A, B, C, D;

end process;

- It is an error if both sensitivity list and wait statement are present within a process.

3 Basic forms of the wait statement

wait on sensitivity list;

wait until boolean-expression;

wait for time-expression;

wait on sensitivity list until boolean expression
for time expression;

eg:

wait on A, B, C;

wait until A = B;

wait for 10ns;

wait on clock for 20ns;

wait for 0 :

wait for 0ns;

It means wait for one delta delay.

eg:

process

begin

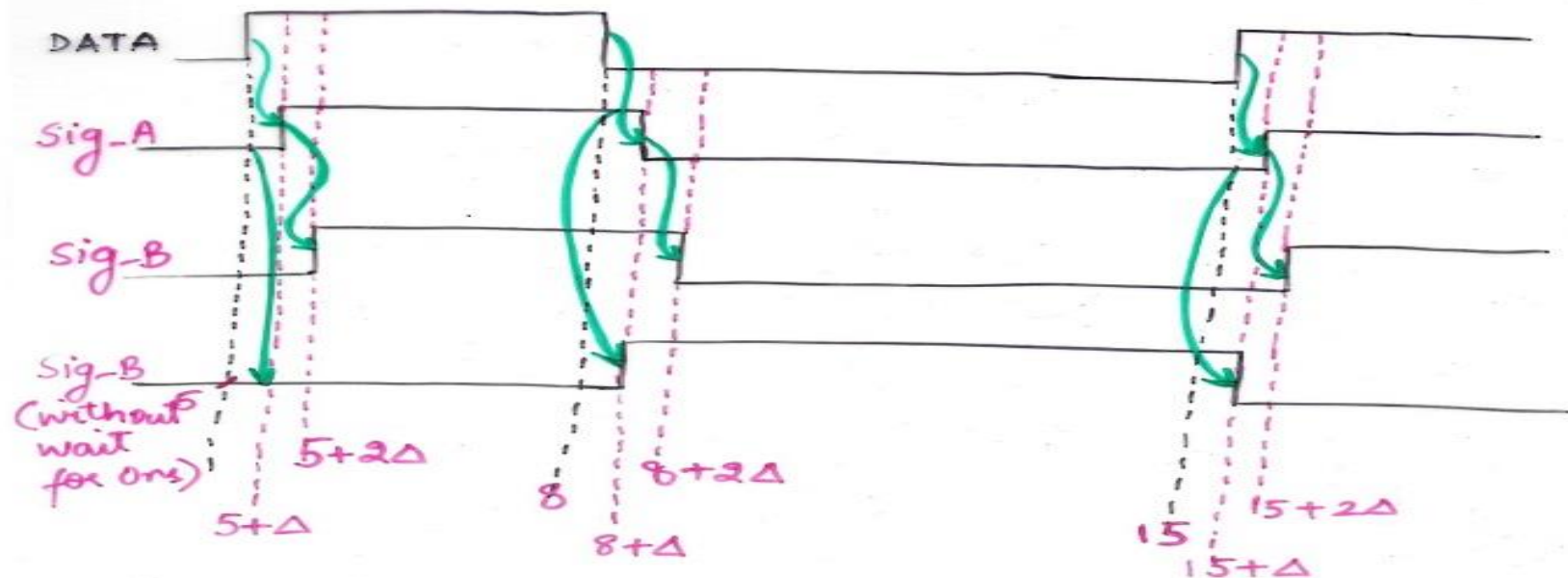
wait on DATA;

sig-A <= DATA;

wait for 0ns;

sig-B <= sig-A;

end process;



wait until:

eg: wait until clock = '1';

if statement:

```

if boolean_expression then
    sequential_statements
{ elsif boolean_expression then
    sequential_statements }
{ else sequential_statements }
end if;
  
```

eg: entity NOR2 is
 port (A, B : in BIT; z : out BIT);
 end NOR2;

Architecture NOR2 of NOR2 is

```
begin
  process (A, B)
    CONSTANT RISE_TIME : TIME := 10 ns;
    CONSTANT FALL_TIME : TIME := 8 ns;
  begin
    if (A nor B) = '1' then
      z <= '1' after RISE_TIME;
    else
      z <= '0' after FALL_TIME;
    endif;
  end process;
end NOR2;
```

Case statement :

- Case statement selects one of the branches for execution based on the value of the expression.

Format for case statement is

BM-7

Case expression is

when choices $\Rightarrow$ sequential statements -- branch #1
when choices $\Rightarrow$ seqⁿ statements -- branch #2
when others $\Rightarrow$ seqⁿ statements -- Last Branch
end case;

- Expression value must be of discrete type or 1D array type.
- choices may be expressed as a single values or as a range of values by using 1 (vertical bar).
- All possible values of expression must be covered in the case exactly once.

eg:

type WEEK_DAY is (MON, TUE, WED, THU, FRI, SAT, SUN);

type DOLLARS is range 0 to 10;

Variable DAY : WEEK_DAY;

Variable POCKET_MONEY : DOLLARS;

Case DAY is

when TUE $\Rightarrow$ POCKET_MONEY := 6;

when MON | WED $\Rightarrow$ POCKET_MONEY := 2;

when FRI to SUN $\Rightarrow$ POCKET_MONEY := 7;

when others $\Rightarrow$ POCKET_MONEY := 0;

end case;

Null statement:

does not cause any action to take place; execution continues with the next statement.

- Used in if or case statement.

Loop statement:

It is used to iterate through a set of sequential statements.

Syntax is

```
[Loop-label:] iteration_scheme loop
    sequential_statements
end loop [loop-label];
```

2 Types of iteration schemes:

First is FOR iteration scheme.

FOR identifier in range

eg:

```
FACTORIAL := 1;
FOR NUMBER in 2 to N loop
    FACTORIAL := FACTORIAL * NUMBER;
end loop;
```

- * NUMBER implicitly declared to be integer.
- * LOOP IDENTIFIER → No explicit declaration necessary.
- can't be assigned any value in loop.

- The range in for loop can also be range of enumeration type. eg:

type HEXA is ('0', '1', '2', '3', '4', '5', '6', '7', '8',
 '9', 'A', 'B', 'C', 'D', 'E', 'F');
...

for NUM in HEXA('9') downto HEXA('0') loop
...
end loop;

Second iteration scheme is while scheme

while Boolean-expression

eg: J := 0; SUM := 10;

WH-LOOP while J < 20 loop

SUM := SUM * 2;

J := J + 3;

end loop;

In third, no iteration scheme is specified.

All statements in the loop body are repeatedly executed until some other action cause the loop to exit.

This action can be caused by an exit statement, a next statement or a return statement.

eg: $SUM := 1; J := 0;$
 L2: Loop
 $J := J + 21;$
 $SUM := SUM * 10;$
 exit when $SUM > 100;$
 end loop L2;

EXIT statement

Syntax is

exit [loop label] [when condition];

- If no loop label specified, the innermost loop is exited.

eg: ① $SUM := 1; J := 0;$
 L3: Loop
 $J := J + 21;$
 $SUM := SUM * 10;$
 if $SUM > 100$ then
 exit L3;
 end if;
 end loop L3;

② Loop
 wait on A, B;
 exit when $A = B;$
 end loop;

equivalent to wait until $A = B;$

Next statement: • Used only inside a loop.

- replaces keyword exit.

Syntax is

next {loop label} (when condition);

- Results in skipping the remaining statements.

eg-

For J in 10 downto 5 loop

if SUM < TOTAL-SUM then

SUM := SUM + 2;

elsif SUM = TOTAL-SUM then

next;

else

null;

endif;

K := K + 1;

end loop;

- when next statement is executed, execution jumps to end of loop, decrements the value of loop identifier J, and resumes loop execution with this new value of J.

Assertion statement:

- Certain statements in VHDL are useful in modeling constraints of an entity.

which type of constraints?

- If we want to check if a signal value lies within specified range.
- To check the setup and hold times for signals arriving at the inputs of an entity.

Syntax:

Assert boolean-expression
 [report string-expression]
 [severity expression];

- If the Boolean expression is false, report message is printed along with the severity level.
- Expression in the severity clause must be a value of type SEVERITY_LEVEL, a predefined enumeration type with values NOTE, WARNING, ERROR & FAILURE.
- Severity level is typically used by a simulator to initiate appropriate actions depending upon its value.
- Default severity level is 'Error'.

Set up time: It is the time required for the input data to settle in before the triggering edge of clock.

Hold time: It is the time required for which the data must remain stable after the triggering edge of clock.

entity DFF is

port (D, CK: in Bit; Q, NOTQ: out bit);

end DFF;

Architecture CHECK_TIMES of DFF is

constant HOLD_TIME: TIME := 5ns;

constant SETUP_TIME: TIME := 3ns;

begin

process (D, CK)

Variable LastEventOnD, LastEventOnCK: Time;

begin

if D'EVENT then

assert NOW = 0ns or (NOW - LastEventOnCK) $\geq$ HOLD_TIME

report "Hold time too short!"

severity failure;

LastEventOnD := NOW;

end if;

Predefined attribute of signal

Pre-defined fn that returns the current simulation time.

- If Assert condition fails, report message is printed and severity level is returned to simulator.
- Default Assert message is 'Assertion Violation'

-- For setup time

if CK = '1' and CK'EVENT then

assert NOW = 0ns or (NOW - LastEventOn D) >=

SET-UP TIME

report "setup time too short!"

severity FAILURE;

LastEventOn CK := NOW;

end if;

if CK = '1' and CK'Event then

Q <= D;

NOT Q <= not D;

end if;

end process;

end CHECK-TIMES;

Report statement

→ can be used to display a message.

Syntax

Report string-expression

[severity expression];

→ must be of predefined

type SEVERITY-LEVEL.

DEFAULT SEVERITY LEVEL IS NOTE

eg: if CLR = 'Z' then
 report "Signal CLR has high impedance value."
 end if;
 if CLK /= '0' and CLK /= '1' then
 report "CLK is neither a '0' nor a '1' !!!"
 severity ERROR;
 end if;

More on signal assignment statement

- Except delta delay, two other models that can be used with signal assignments -
 1. Inertial.
 2. Transport.

More on signal Assignment statement

→ Two other delay models can be used with signal assignments: (a) Inertial delay (b) Transport delay.

Initial delay Model:

- often used in switching circuits.
- Input value must be stable for specified 'pulse rejection limit' before value is allowed to propagate to the O/P.

→ Value appears at output after specified 'inertial delay'.

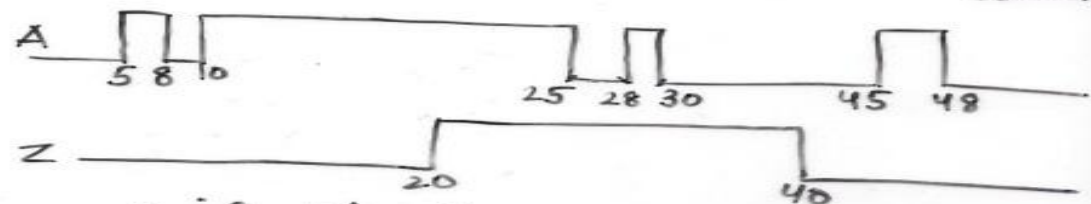
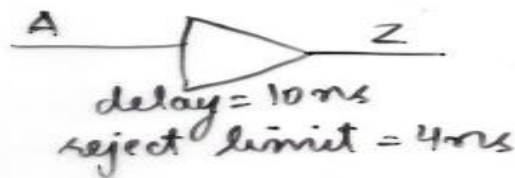
→ If I/P not stable for specified limit, O/P will not change.

Syntax:

signal-object $\Leftarrow$ ([reject pulse-rejection-limit] inertial]
expression after inertial-delay-value;

→ If no pulse rejection limit specified, default value is inertial delay itself.

pulse rejection limit always $\leq$ inertial delay [$\frac{\text{reject}}{\text{inertial}} > 0$].
(+ve value)



$Z \Leftarrow$ reject 4ns inertial after 10ns;

if $Z \leq A$ after 10ns;

Pulse rejection limit is also 10ns, which is inertial delay value.

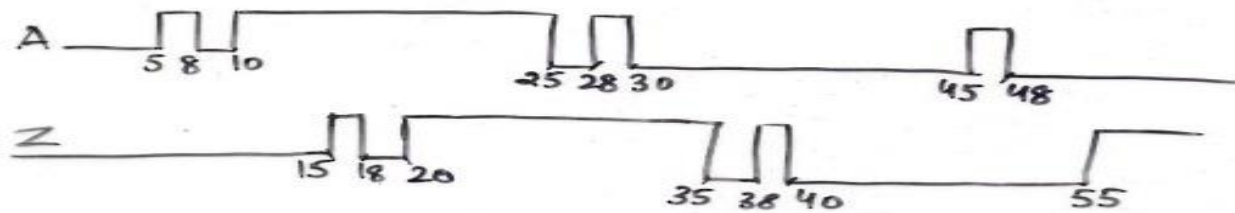
Also, $Z \leq$ inertial A after 10ns; [same meaning as above statement].

Delay model often used to filter out unwanted spikes and transients on signals.

Transport Delay model

- models the delays in Hardware.
- represents pure propagation delay, any change in I/P are transported to O/P, after specified delay.
- To use transport delay model, keyword 'Transport' must be used.

$Z \leq$ Transport A after 10ns;



- Using this, ideal modelling can be obtained.
- spikes are propagated instead of being ignored.

Entity wire14 is
 port (A: in bit; z: out Bit);
 and wire14;

Architecture wire14-transport of wire14 is
 begin
 process(A)
 begin
 Z <= Transport A after 0.1 ms;
 end process;
 end wire14-transport;

Creating signal waveforms:

→ It is possible to assign multiple values to signals, each with different delay value.

eg:-

Phase1 <= '0', '1' after 8ns, '0' after 13ns, '1' after 50ns;

Assume, statement executed at time T,



General syntax:

signal-object <= [transport] [reject pulse rejection limit]

* If no, pulse rejection limit, delay in first waveform-element becomes default limit

inertial] waveform-element, waveform-element, waveform-element, waveform-element; -- any no. of elements.

→ Delays in waveform elements must appear in increasing order.

Signal Drivers:

- Mainly created when more than one assignment to the same signal within a process.

A driver is created for every signal that is assigned a value in a process.

Driver: holds its current value and all its future values, as sequence of one or more transactions.

eg- process
begin

RESET <= 3 after 5ns, 21 after 10ns, 14 after 17ns;
end process;

RESET ←

curr@now	3@T+5ns	21@T+10ns	14@T+17ns
----------	---------	-----------	-----------

- All transactions on drivers are ordered in increasing order of time.
- contains at least one transaction, which could be initial value of signal.
- In RESET, first transaction is current value of signal.
- when simulation time advances to T+5ns, first transaction is deleted, from driver & RESET gets value 3. and so on.

if there is another signal assignment to RESET in same process?

BM-19

→ Signal inside a process has only one driver, transactions of second signal assignment modify the transaction already present on the driver.

Effect of Transport delay on signal Drivers:

eg-
signal RX-DATA: NATURAL;
...
process
begin
...

RX-DATA <= transport 11 after 10ns;
RX-DATA <= transport 20 after 22ns;
RX-DATA <= transport 35 after 18ns;

end process;

* Let statement executed at time, T.

When first signal assignment is executed, transaction 11 @ T + 10ns is added. 11ly for second ∵ (22ns > 10ns)

RX-DATA ←

curr@ now	11@ T + 10ns	20@ T + 22ns
-----------	--------------	--------------

A new transaction will delete all transactions sitting on a driver are to occur at or later than the delay of new transaction.

RX-DATA ←

curr@ now	11@ T + 10ns	35@ T + 18ns
-----------	--------------	--------------

eg:- process
begin
...

DATA_BUS <= transport X"01" after 5ns, X"FA" after 10ns,
X"E8" after 15ns;

DATA_BUS <= transport X"B5" after 12ns;

end process;

After first signal Assignment:

DATA_BUS ←

curr@now	X"01"@T+ _{5ns}	X"FA"@T+10ns	X"E8"@T+15ns
----------	-------------------------	--------------	--------------

When second signal assignment:

DATA_BUS ←

curr@now	X"01"@T+5ns	X"FA"@T+10ns	X"B5"@T+12ns
----------	-------------	--------------	--------------

Rules for adding transactions:

1. All transactions on the device that occur at or after the delay time of first new transaction are deleted.
2. All the new transactions are added at the end of the device.

Effect of inertial delay on signal drivers:

- Delay value affect the deletion & addition of transactions
- Rule 1 of transport delay applies to this also.
- Other Rule,

Existing transactions that fall within the window of the time of the first new transaction, say F and (F - pulse rejection limit), are checked for their values.

- If values are different, These already existing transactions are deleted.
- Otherwise, they are retained.

eg: process

begin

Tx-data <= 11 after 10ns; -- stmt 1

Tx-data <= reject 15ns inertial 22 after 20ns;

Tx-data <= 33 after 15ns; -- stmt 3 -- stmt 2

wait;

end process;

For stmt 1:

Tx-data ←

curr@now	11 @ 10ns
----------	-----------

For stmt 1 & 2:

Tx-data ←

curr@now	22 @ 20ns
----------	-----------

∴ Here, $F = 20\text{ns}$, (F - Pulse rejection limit)

$$20 - 15 = 5\text{ns}.$$

Values lies b/w 5ns and 20ns. (-- stmt 1)

check for value.

Value for stmt 1 = 11, for stmt 2 = 22 ∴ stmt 1 deleted.

For stmt 3 added:

$Tx_data \leftarrow$

curr@now	33 @ 15 ns
----------	------------

Acc. to Rule 1 of transport delay.

eg2: process
begin

Adder_bus <= 1 after 5ns, 21 after 9ns, 6 after 10ns,
12 after 19ns; -- stmt1

Adder_bus <= reject 4ns inertial 6 after 12ns, 20 after
19ns; -- stmt2

wait;
end process;

For stmt 1:

Adder_bus <=

curr@now	1 @ 5ns	21 @ 9ns	6 @ 10ns	12 @ 19ns
----------	---------	----------	----------	-----------

For stmt 2 added: According to Rule 1 of transport delay,
12 @ 19ns from stmt1 gets deleted.

Here, $F = 12\text{ns}$, $(12 - 4) = 8\text{ns}$

B/w 8ns & 12ns two values, 21 @ 9ns & 6 @ 10ns
21 @ 9ns gets deleted $\therefore$ value @ 12ns is '6'.

Adder_bus <=

curr@now	1 @ 5ns	6 @ 10ns	6 @ 12ns	20 @ 19ns
----------	---------	----------	----------	-----------

Multiple Processes:

B.M.23

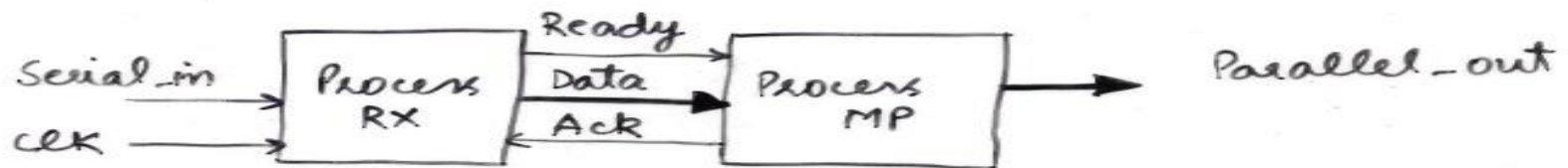
- Possible to have more than one process in an architecture body.
- Processes within architecture body communicate with each other using signals that are visible to all the processes.
(shared variables can be used).

eg- Two interacting processes:

RX: A Receiver

MP: Microprocessor

Block Diagram for Two processes:



entity interacting is

port (Serial_in, clk : in bit;

Parallel_out : out bit_vector (0 to 7));

end interacting;

Architecture Processes of interacting is

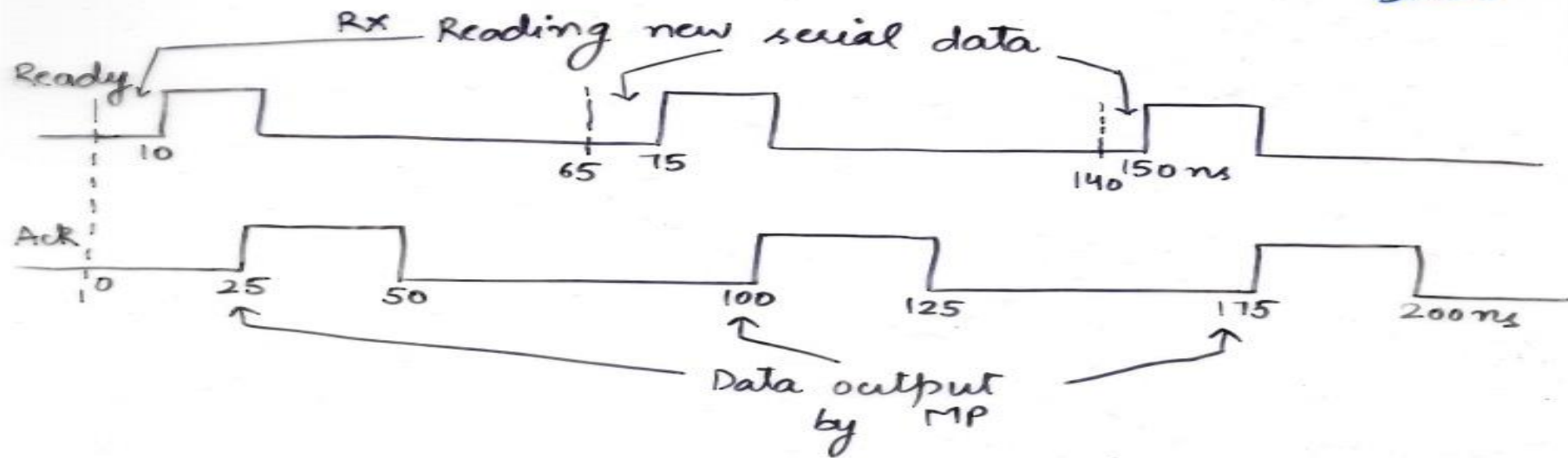
```

signal Ready, Ack: bit;
signal Data: bit_vector (0 to 7);
begin
  RX: Process
  begin
    READ_WORD (SERIAL_IN, CLK, DATA);
    Ready <= '1';
    wait until Ack = '1';
    Ready <= '0';
    wait for 40 ns;
  end process RX;

  MP: Process
  begin
    wait for 25 ns;
    Parallel_out <= Data;
    Ack <= '1', '0' after 25 ns;
    wait until Ready = '1';
  end process MP;
end Processes;

```

→ Possible to drive more than one process to drive the same signal.



Handshaking protocol between the ^{two} processors.

Postponed Process

BM26

...
begin

...
end postponed Process

- Such a process executes only at the end of all deltas of the simulation time in which event triggered the process to resume execution.

postponed process (A, B)

begin

$Z \leq A \text{ nor } B \text{ after } 8\text{ns};$

end postponed process;

- If event occurs on signal A, B, say, at times 5ns , $5+\Delta$, $5+4\Delta$, 10ns , $10+3\Delta$, 16ns and $16+\Delta$, then the postponed process will execute only after all the deltas at time 5ns , all the deltas at time 10ns and all the deltas at time 16ns .

Ex: 1:

BM-ex1

signal A, Z : INTEGER;

...
PZ : Process(A)

Variable V1, V2 : INTEGER;

begin

V1 := A - V2;

Z <= -V1;

V2 := Z + V1 * 2;

end process PZ;

Ex: 2:

if SUM <= 100 then

SUM := SUM + 10;

end if;

Ex: 3:

if NICKEL_IN then

DEPOSITED <= TOTAL - 10;

elsif DIME_IN then

DEPOSITED <= TOTAL - 15;

elsif QUATER_IN then

DEPOSITED <= TOTAL - ERROR;

end if;

Ex: 4:

if CTRL1 = '1' then

if CTRL2 = '0' then

MUX-OUT <= "0010";

else

MUX-OUT <= "0001";

endif;

Ex 5: entity MUX is
port (A, B, C, D : in std_logic;
 CTRL : in std_logic_vector (0 to 1);
 Z : out std_logic);
end MUX;

BM-
CX2

Architecture Mux_beh of Mux is

constant Mux_delay : Time := 10 ns;
begin

PMUX: process (A, B, C, D, CTRL)

Variable Temp: std_logic;

begin

case CTRL is

when "00" => Temp := A;

when "01" => Temp := B;

when "10" => Temp := C;

when "11" => Temp := D;

when others => Temp := 'X';

end case;

Z <= Temp after Mux_delay;

end process PMUX;

end Mux_beh;

Ex 6:

L4: for K in 10 downto 1 loop

---- stmt1

L5: loop

---- stmt2

next L4 when WR_DONE = '1';

---- stmt3

end loop L5;

---- stmt4

end loop L4;

Process using Variables:

entity dummy is
end dummy;

architecture var of dummy is
signal trigger, sum: integer := 0;

begin

process

variable var1: integer := 1;

variable var2: integer := 2;

variable var3: integer := 3;

begin

wait on trigger;

var1 := var2 + var3;

var2 := var1;

var3 := var2;

sum <= var1 + var2 + var3;

end process;

end var;

Process using signals

entity dummy is
end dummy;

architecture sig of dummy is
signal trigger, sum : integer := 0;

signal sig1 : integer := 1;

signal sig2 : integer := 2;

signal sig3 : integer := 3;

begin

Process

begin

wait on trigger;

sig1 <= sig2 + sig3;

sig2 <= sig1;

sig3 <= sig2;

sum <= sig1 + sig2 + sig3;

end process;

end sig;